maix.display

maix.display module, control display device and show image on it

> You can use `maix.display` to access this module with MaixPy
> This module is generated from MaixPy and MaixCDK

1. Module

No module

2. Enum

3. Variable

4. Function

4.1. send_to_maixvision

```python
def send_to_maixvision(img: maix.image.Image) -> None
```

Send image to MaixVision work station if connected.\nIf you want to debug your program an don't want to initialize display, use this method.

item	description
param	**img**: image to send, image.Image object

> C++ defination code:
>
> ```
> void send_to_maixvision(image::Image &img)
> ```

4.2. set_trans_image_quality

```
def set_trans_image_quality(value: int) -> None
```

Set image transport quality(only for JPEG)

item	description
param	**quality**: default 95, value from 51 ~ 100

> C++ defination code:
>
> ```
> void set_trans_image_quality(const int value)
> ```

5. Class

5.1. Display

Display class

> C++ defination code:
>
> ```
> class Display
> ```

5.1.1. __init__

```
def __init__(self, width: int = -1, height: int = -1, format: maix.image.Format = ..., device: str = '', open: bool = True) -> None
```

Construct a new Display object

item	description
type	func
param	**width**: display width, by default(value is -1) means auto detect, if width > max device supported width, will auto set to max device supported width **height**: display height, by default(value is -1) means auto detect, if height > max device supported height, will auto set to max device supported height **device**: display device name, you can get devices by list_devices method, by default(value is NULL(None in MaixPy)) means the first device **open**: If true, display will

item	description
	automatically call open() after creation. default is true.
static	False

> C++ defination code:
>
> ```
> Display(int width = -1, int height = -1, image::Format format = image::FMT_RGB888, const std::string &device = "", bool open = true)
> ```

5.1.2. width

```
def width(self) -> int
```

Get display width

item	description
type	func
return	width
static	False

> C++ defination code:
>
> ```
> int width()
> ```

5.1.3. height

```
def height(self) -> int
```

Get display height

item	description
type	func
param	**ch**: channel to get, by default(value is 0) means the first channel
return	height
static	False

> C++ defination code:
>
> ```
> int height()
> ```

5.1.4. size

```
def size(self) -> list[int]
```

Get display size

item	description
type	func
param	**ch**: channel to get, by default(value is 0) means the first channel
return	size A list type in MaixPy, [width, height]
static	False

> C++ defination code:

```
std::vector<int> size()
```

5.1.5. format

```
def format(self) ->
maix.image.Format
```

Get display format

item	description
type	func
return	format
static	False

> C++ defination code:
>
> ```
> image::Format format()
> ```

5.1.6. open

```
def open(self, width: int = -1,
height: int = -1, format:
maix.image.Format = ...) ->
maix.err.Err
```

open display device, if already opened, will return err.ERR_NONE.

item	description
type	func

item	description
param	**width**: display width, default is -1, means auto, mostly means max width of display support **height**: display height, default is -1, means auto, mostly means max height of display support **format**: display output format, default is RGB888
return	error code
static	False

> C++ defination code:
>
> ```
> err::Err open(int width = -1, int height = -1, image::Format format = image::FMT_INVALID)
> ```

5.1.7. close

```
def close(self) -> maix.err.Err
```

close display device

item	description
type	func
return	error code
static	False

> C++ defination code:

```
err::Err close()
```

5.1.8. add_channel

```
def add_channel(self, width: int = -1, height: int = -1, format: maix.image.Format = ..., open: bool = True) -> Display
```

Add a new channel and return a new Display object, you can use close() to close this channel.

item	description
type	func
attention	If a new disp channel is created, it is recommended to set fit=image::FIT_COVER or fit=image::FIT_FILL when running show for the main channel, otherwise the display of the new disp channel may be abnormal.
param	**width**: display width, default is -1, means auto, mostly means max width of display support. Maximum width must not exceed the main channel. **height**: display height, default is -1, means auto, mostly means max height of display support. Maximum height must not exceed the main channel.

item	description
	format: display output format, default is FMT_BGRA8888 **open**: If true, display will automatically call open() after creation. default is true.
return	new Display object
static	False

> C++ defination code:
>
> ```
> display::Display *add_channel(int width = -1, int height = -1, image::Format format = image::FMT_BGRA8888, bool open = true)
> ```

5.1.9. is_opened

```
def is_opened(self) -> bool
```

check display device is opened or not

item	description
type	func
return	opened or not, bool type
static	False

> C++ defination code:

```
bool is_opened()
```

5.1.10. is_closed

```
def is_closed(self) -> bool
```

check display device is closed or not

item	description
type	func
return	closed or not, bool type
static	False

> C++ defination code:
>
> ```
> bool is_closed()
> ```

5.1.11. show

```
def show(self, img: maix.image.Image, fit: maix.image.Fit = ...) -> maix.err.Err
```

show image on display device, and will also send to MaixVision work station if connected.

item	description
type	func

item	description
param	**img**: image to show, image.Image object, if the size of image smaller than display size, will show in the center of display; if the size of image bigger than display size, will auto resize to display size and keep ratio, fill blank with black color. **fit**: image in screen fit mode, by default(value is image.FIT_CONTAIN), @see image.Fit for more details e.g. image.FIT_CONTAIN means resize image to fit display size and keep ratio, fill blank with black color.
return	error code
static	False

> C++ defination code:
>
> ```
> err::Err show(image::Image &img, image::Fit fit = image::FIT_CONTAIN)
> ```

5.1.12. device

```
def device(self) -> str
```

Get display device path

item	description
type	func
return	display device path
static	False

> C++ defination code:
>
> ```
> std::string device()
> ```

5.1.13. set_backlight

```
def set_backlight(self, value: float) -> None
```

Set display backlight

item	description
type	func
param	**value**: backlight value, float type, range is [0, 100]
static	False

> C++ defination code:
>
> ```
> void set_backlight(float value)
> ```

5.1.14. get_backlight

```
def get_backlight(self) -> float
```

Get display backlight

item	description
type	func
return	value backlight value, float type, range is [0, 100]
static	False

> C++ defination code:
>
> ```
> float get_backlight()
> ```

5.1.15. set_hmirror

```
def set_hmirror(self, en: bool) ->
maix.err.Err
```

Set display mirror

item	description
type	func
param	**en**: enable/disable mirror
static	False

> C++ defination code:
>
> ```
> err::Err set_hmirror(bool en)
> ```

5.1.16. set_vflip

```
def set_vflip(self, en: bool) -> maix.err.Err
```

Set display flip

item	description
type	func
param	**en**: enable/disable flip
static	False

> C++ defination code:
>
> ```
> err::Err set_vflip(bool en)
> ```